

Un algorithme pour la dualité d'Aubert–Zelevinsky

Thomas Lanard
(en collaboration avec Alberto Mínguez)

23 avril 2026

Séminaire Groupes, Algèbre et Géométrie, Poitiers

Obtenir les diapos :



[https://www.thomaslanard.com/
slides/AZ.pdf](https://www.thomaslanard.com/slides/AZ.pdf)

Une involution pour les groupes réductifs finis

Les caractères unipotents de $\mathrm{GL}_n(\mathbb{F}_q)$ sont paramétrés par les partitions de n .

Il existe une involution naturelle donnée par la transposée des partitions.

Théorème (Dualité d'Alvis–Curtis)

Cette involution est la dualité d'Alvis–Curtis pour $\mathrm{GL}_n(\mathbb{F}_q)$.

Une involution pour les groupes réductifs finis

Les caractères unipotents de $\mathrm{GL}_n(\mathbb{F}_q)$ sont paramétrés par les partitions de n .

Il existe une involution naturelle donnée par la transposée des partitions.

Théorème (Dualité d'Alvis–Curtis)

Cette involution est la dualité d'Alvis–Curtis pour $\mathrm{GL}_n(\mathbb{F}_q)$.

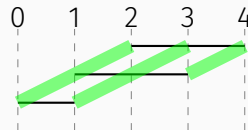
$\mathrm{GL}_n(\mathbb{Q}_p)$		$\mathrm{GL}_n(\mathbb{F}_q)$
Aubert–Zelevinsky	$\longleftrightarrow$	Alvis–Curtis
Multisegments	$\longleftrightarrow$	Partitions
Mœglin–Waldspurger	$\longleftrightarrow$	Transposée

- Involution d'Aubert–Zelevinsky
- Le cas de GL_n
- Le cas des groupes classiques
- Implémentation
- Machine Learning

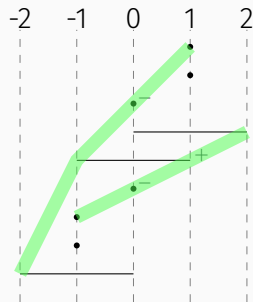
- Involution d'Aubert–Zelevinsky
- Le cas de GL_n
- Le cas des groupes classiques
- Implémentation
- Machine Learning

$$\begin{array}{ccc} \mathrm{Irr}(G) & \longrightarrow & \mathrm{Irr}(G) \\ \pi & \longmapsto & \hat{\pi} \end{array}$$

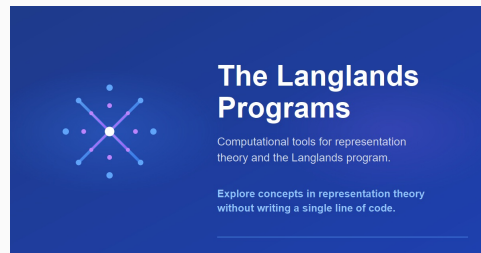
- Involution d'Aubert-Zelevinsky
- Le cas de GL_n
- Le cas des groupes classiques
- Implémentation
- Machine Learning



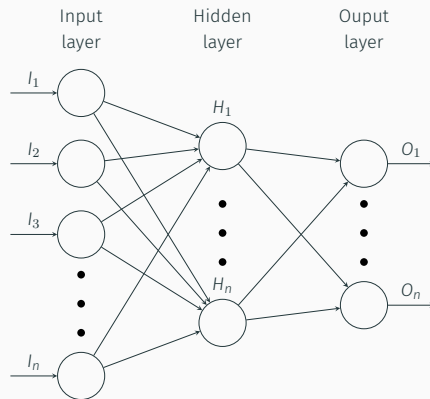
- Involution d'Aubert-Zelevinsky
- Le cas de GL_n
- Le cas des groupes classiques
- Implémentation
- Machine Learning



- Involution d'Aubert–Zelevinsky
- Le cas de GL_n
- Le cas des groupes classiques
- Implémentation
- Machine Learning



- Involution d'Aubert-Zelevinsky
- Le cas de GL_n
- Le cas des groupes classiques
- Implémentation
- Machine Learning



Involution d'Aubert-Zelevinsky

Contexte :

- F un corps p -adique de norme $|\cdot|$
- G un groupe p -adique

Représentations :

- $\text{Rep}(G)$ = représentations lisses complexes de longueur finie
- $\text{Irr}(G)$ = représentations irréductibles
- $\text{Cusp}(G)$ = représentations cuspidales irréductibles

Foncteurs : Pour un sous-groupe parabolique $P = MN$,

$$\mathbf{i}_P^G: \text{Rep}(M) \rightarrow \text{Rep}(G), \quad \mathbf{r}_P^G: \text{Rep}(G) \rightarrow \text{Rep}(M)$$

(induction et restriction paraboliques normalisées).

Opérateur sur le groupe de Grothendieck

Soit $R(G)$ le groupe de Grothendieck de $\text{Rep}(G)$.

Définition

Définissons l'opérateur

$$D_G: R(G) \longrightarrow R(G)$$

par

$$D_G(\pi) := \sum_{\substack{P=MU \\ \text{standard}}} (-1)^{\dim(A_M)} i_P^G(r_P^G(\pi))$$

où A_M est le tore déployé maximal dans le centre de M .

Théorème (Aubert 1995)

Si $\pi \in \text{Irr}(G)$, il existe $\varepsilon \in \{\pm 1\}$ tel que

$$\widehat{\pi} := \varepsilon D_G(\pi) \in \text{Irr}(G).$$

Cela définit une application, appelée **involution d'Aubert–Zelevinsky** :

$$\begin{array}{ccc} \text{Irr}(G) & \longrightarrow & \text{Irr}(G) \\ \pi & \longmapsto & \widehat{\pi} \end{array}$$

- L'application $\pi \mapsto \hat{\pi}$ est une **involution** sur $\text{Irr}(G)$.

- L'application $\pi \mapsto \hat{\pi}$ est une **involution** sur $\text{Irr}(G)$.
- Si π est **cuspidale**, alors $\hat{\pi} = \pi$.

- L'application $\pi \mapsto \widehat{\pi}$ est une **involution** sur $\text{Irr}(G)$.
- Si π est **cuspidale**, alors $\widehat{\pi} = \pi$.
- Elle échange la **représentation triviale** et la **Steinberg** :

$$\widehat{\mathbf{1}_G} = \text{St}_G \quad \text{et} \quad \widehat{\text{St}_G} = \mathbf{1}_G.$$

Le cas de GL_n

On note par $\times$ l'induction parabolique pour GL_n .

Si $\pi_i \in \text{Rep}(GL_{n_i})$, alors

$$\pi_1 \times \pi_2 = i_{GL_{n_1} \times GL_{n_2}}^{GL_{n_1+n_2}} (\pi_1 \boxtimes \pi_2).$$

On écrit $\text{Cusp}(GL) = \bigsqcup_{n \geq 1} \text{Cusp}(GL_n)$ pour l'ensemble de toutes les représentations cuspidales.

Définition

Un **segment** est un sous-ensemble de $\text{Cusp}(\text{GL})$ de la forme

$$\Delta = \{\rho| \cdot |^x, \rho| \cdot |^{x+1}, \dots, \rho| \cdot |^y\}$$

où $\rho \in \text{Cusp}(\text{GL})$, et $x \leq y$ avec $y - x \in \mathbb{N}$.

On utilise la notation $\Delta = [x, y]_\rho$.

Exemple : $\Delta = [0, 2]_\rho = \{\rho, \rho| \cdot |, \rho| \cdot |^2\}$

Opérations sur les segments

Pour un segment $\Delta = [x, y]_\rho$ (avec ρ unitaire) :

- $e(\Delta) = y$ (l'**extrémité** de Δ)
- $b(\Delta) = x$ (la **base** de Δ)
- $c(\Delta) = \frac{x+y}{2}$ (le **centre** de Δ)

On définit $\Delta^- = [x, y - 1]_\rho$ (on enlève la dernière cuspidale).

Exemple : Pour $\Delta = [0, 2]_\rho$:

- $e(\Delta) = 2, b(\Delta) = 0, c(\Delta) = 1$
- $\Delta^- = [0, 1]_\rho = \{\rho, \rho | \cdot |\}$

Définition

Un **multisegment** est une somme formelle finie de segments :

$$\mathfrak{m} = \Delta_1 + \cdots + \Delta_r.$$

On note par Mult l'ensemble de tous les multisegments.

Exemple : $\mathfrak{m} = [2, 3]_\rho + [2, 3]_\rho + [0, 2]_\rho + [2, 2]_\rho$ est un multisegment.

Théorème (Zelevinsky 1980)

À chaque segment $\Delta = [x, y]_\rho$, on peut associer deux représentations irréductibles $Z(\Delta)$ et $L(\Delta)$:

- $Z(\Delta)$ est l'unique **sous-représentation** irréductible de la représentation induite

$$\rho| \cdot |^x \times \cdots \times \rho| \cdot |^y$$

- $L(\Delta)$ est l'unique **quotient** irréductible de cette induite.

Théorème (Zelevinsky 1980)

À chaque multisegment $\mathbf{m} = \Delta_1 + \cdots + \Delta_r$, tel que $c(\Delta_1) \geq \cdots \geq c(\Delta_r)$, on associe :

- $Z(\mathbf{m})$: l'unique sous-représentation irréductible de $Z(\Delta_1) \times \cdots \times Z(\Delta_r)$
- $L(\mathbf{m})$: l'unique quotient irréductible de $L(\Delta_1) \times \cdots \times L(\Delta_r)$

$Z(\mathbf{m})$ et $L(\mathbf{m})$ ne dépendent que de $\mathbf{m}$.

Les applications $\mathbf{m} \mapsto Z(\mathbf{m})$ et $\mathbf{m} \mapsto L(\mathbf{m})$ donnent des **bijections**

$$\text{Mult} \xrightarrow{\sim} \text{Irr}(\text{GL}).$$

Exemples :

Multisegment $\mathbf{m}$	$Z(\mathbf{m})$	$L(\mathbf{m})$
$\left[-\frac{1}{2}, \frac{1}{2}\right]$	$\mathbf{1}_{\text{GL}_2}$	St_{GL_2}
$\left[-\frac{1}{2}, -\frac{1}{2}\right] + \left[\frac{1}{2}, \frac{1}{2}\right]$	St_{GL_2}	$\mathbf{1}_{\text{GL}_2}$

Exemples :

Multisegment $\mathbf{m}$	$Z(\mathbf{m})$	$L(\mathbf{m})$
$[-\frac{1}{2}, \frac{1}{2}]$	$\mathbf{1}_{\text{GL}_2}$	St_{GL_2}
$[-\frac{1}{2}, -\frac{1}{2}] + [\frac{1}{2}, \frac{1}{2}]$	St_{GL_2}	$\mathbf{1}_{\text{GL}_2}$

Lien entre $Z(\mathbf{m})$ et $L(\mathbf{m})$

$$\widehat{Z(\mathbf{m})} = L(\mathbf{m})$$

Question

Étant donné un multisegment $\mathbf{m}$, trouver $\mathbf{m}^t$ tel que

$$\widehat{Z(\mathbf{m})} = Z(\mathbf{m}^t).$$

Un algorithme explicite a été donné par [Mœglin et Waldspurger](#).

L'algorithme fonctionne indépendamment sur chaque **droite cuspidale**.

Définition

Une **droite cuspidale** est

$$\mathbb{Z}_\rho = \{\rho | \cdot |^n, n \in \mathbb{Z}\}$$

pour une représentation cuspidale fixée $\rho \in \text{Cusp}(\text{GL})$.

Soit Mult_ρ l'ensemble des multisegments supportés sur $\mathbb{Z}_\rho$.

Alors $\text{Mult} = \bigoplus_\rho \text{Mult}_\rho$ (où l'on somme sur les droites cuspidales), et cette décomposition est compatible avec la dualité :

$$\mathfrak{m} = \sum \mathfrak{m}_\rho \implies \mathfrak{m}^t = \sum \mathfrak{m}_\rho^t$$

Définition (Ordre sur les segments)

Pour des segments sur une même droite cuspidale :

$$[x, y]_{\rho} \leq [x', y']_{\rho} \quad \text{si} \quad \begin{cases} x < x' \\ \text{ou } x = x' \text{ et } y \geq y' \end{cases}$$

 Ce n'est PAS l'ordre lexicographique !

Exemple : $[0, 2]_{\rho} \leq [1, 6]_{\rho} \leq [1, 3]_{\rho}$

Soit $\mathbf{m} \in \text{Mult}_\rho$ un multisegment.

Étape 1 : Trouver $e_{\max} = \max\{e(\Delta) \mid \Delta \in \mathbf{m}\}$.

Étape 2 : Soit Δ_1 le plus grand segment (pour l'ordre $\leq$) avec $e(\Delta_1) = e_{\max}$.

Étape 3 : Définir de façon inductive $\Delta_1 \geq \Delta_2 \geq \dots \geq \Delta_l$ où Δ_{j+1} est le plus grand segment tel que :

- $\Delta_{j+1} \leq \Delta_j$
- $e(\Delta_{j+1}) = e(\Delta_j) - 1$

(on s'arrête s'il n'existe plus un tel segment).

L'algorithme de Mœglin–Waldspurger

Après avoir trouvé $\Delta_1, \dots, \Delta_l$, on définit :

- Le **premier segment du dual** :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

- Un **nouveau multisegment** :

$$\mathfrak{m}^\# = \mathfrak{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

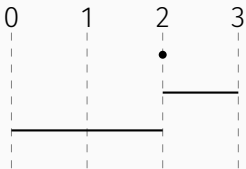
Théorème (Mœglin–Waldspurger 1986)

Le multisegment dual est donné de façon récursive par :

$$\mathfrak{m}^t = \Delta'_1 + (\mathfrak{m}^\#)^t.$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$\mathbf{m}^t = \dots$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

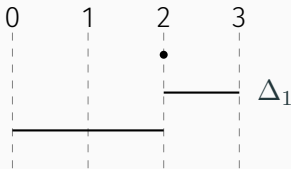
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = \dots$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

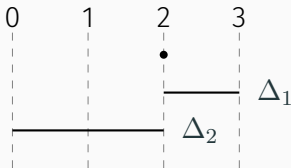
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = \dots$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

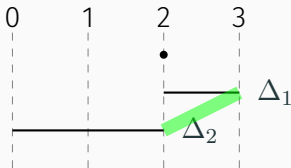
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = [2, 3] + \dots$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

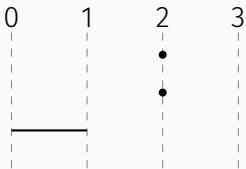
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = [2, 3] + \dots$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

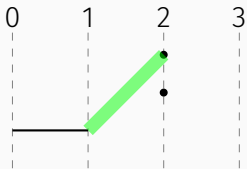
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = [2, 3] + [1, 2] + \dots$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

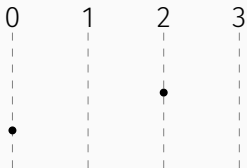
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = [2, 3] + [1, 2] + \dots$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

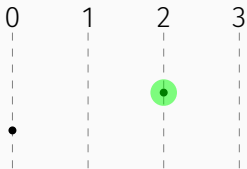
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = [2, 3] + [1, 2] + [2, 2] + \dots$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

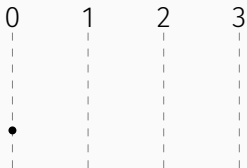
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = [2, 3] + [1, 2] + [2, 2] + \dots$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

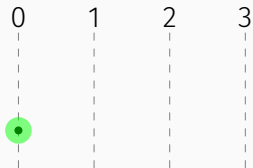
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = [2, 3] + [1, 2] + [2, 2] + [0, 0]$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

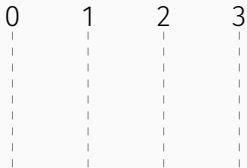
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Exemple 1

Calcul du dual de $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Multisegment dual :

$$\mathbf{m}^t = [2, 3] + [1, 2] + [2, 2] + [0, 0]$$

Rappel : algorithme MW

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ plus grand segment avec $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductivement : Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

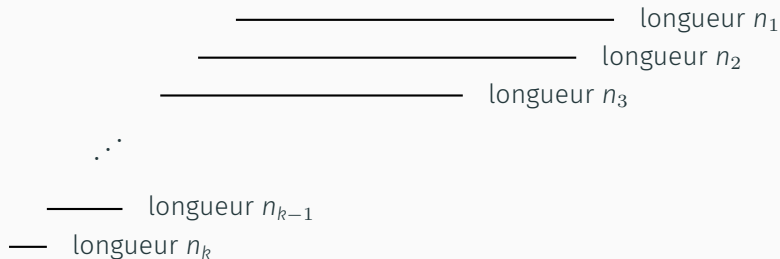
Résultat :

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

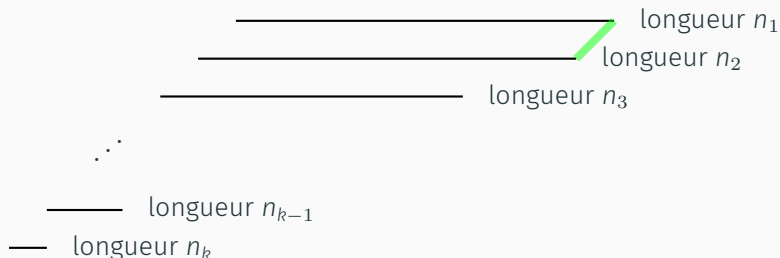
Exemple 2 : Transposée d'une partition

Soit $n_1 \geq \dots \geq n_k$ une partition. Considérons le multisegment $\mathbf{m}$ avec des segments de longueurs $n_1, \dots, n_k$ et des bases décroissants par 1 :



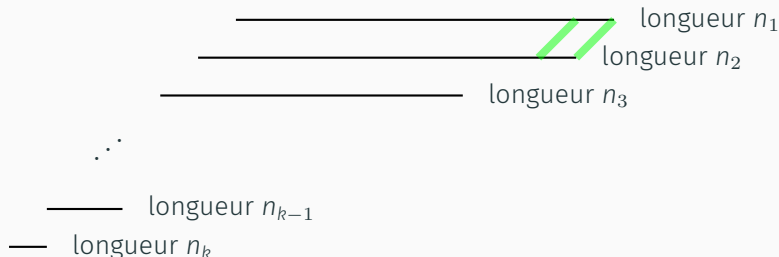
Exemple 2 : Transposée d'une partition

Soit $n_1 \geq \dots \geq n_k$ une partition. Considérons le multisegment $\mathbf{m}$ avec des segments de longueurs $n_1, \dots, n_k$ et des bases décroissants par 1 :



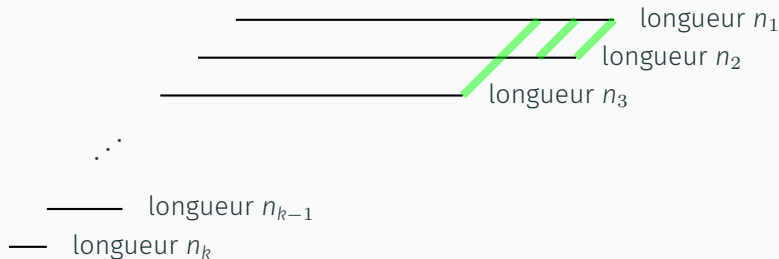
Exemple 2 : Transposée d'une partition

Soit $n_1 \geq \dots \geq n_k$ une partition. Considérons le multisegment $\mathbf{m}$ avec des segments de longueurs $n_1, \dots, n_k$ et des bases décroissants par 1 :



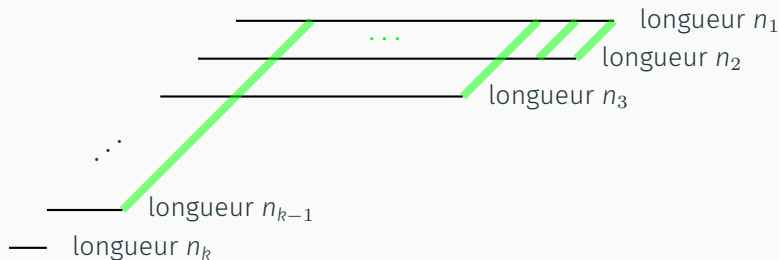
Exemple 2 : Transposée d'une partition

Soit $n_1 \geq \dots \geq n_k$ une partition. Considérons le multisegment $\mathbf{m}$ avec des segments de longueurs $n_1, \dots, n_k$ et des bases décroissants par 1 :



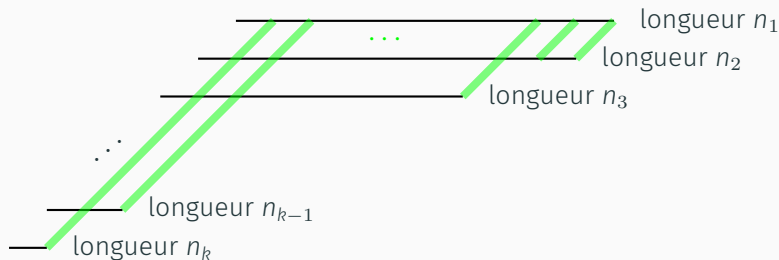
Exemple 2 : Transposée d'une partition

Soit $n_1 \geq \dots \geq n_k$ une partition. Considérons le multisegment $\mathbf{m}$ avec des segments de longueurs $n_1, \dots, n_k$ et des bases décroissants par 1 :



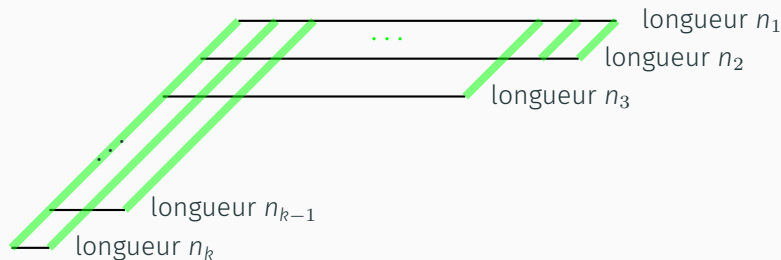
Exemple 2 : Transposée d'une partition

Soit $n_1 \geq \dots \geq n_k$ une partition. Considérons le multisegment $\mathbf{m}$ avec des segments de longueurs $n_1, \dots, n_k$ et des bases décroissants par 1 :



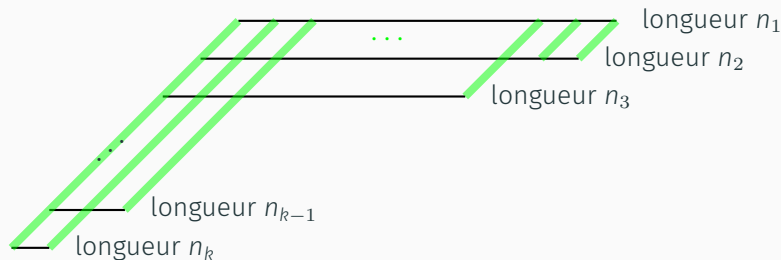
Exemple 2 : Transposée d'une partition

Soit $n_1 \geq \dots \geq n_k$ une partition. Considérons le multisegment $\mathbf{m}$ avec des segments de longueurs $n_1, \dots, n_k$ et des bases décroissants par 1 :



Exemple 2 : Transposée d'une partition

Soit $n_1 \geq \dots \geq n_k$ une partition. Considérons le multisegment $\mathfrak{m}$ avec des segments de longueurs $n_1, \dots, n_k$ et des bases décroissants par 1 :



Alors $\mathfrak{m}^t$ correspond à la **partition transposée** (n_i^*) .

Le cas des groupes classiques

Classification des représentations irréductibles

On considère maintenant $G = \mathrm{Sp}_{2n}(F)$ ou $\mathrm{SO}_{2n+1}(F)$.

Classification par la correspondance de Langlands locale

Les représentations irréductibles de G sont paramétrées par des **données de Langlands**

$$\pi = L(\mathfrak{m}, \phi, \eta)$$

Composants des données de Langlands :

- **Multisegment** : $\mathfrak{m} = \Delta_1 + \cdots + \Delta_r$, où $c(\Delta_i) < 0$
- **Paramètre de Langlands** : $\phi = \bigoplus_{i=1}^k \rho_i \boxtimes S_{a_i}$, où $\rho_i \in \mathrm{Cusp}(\mathrm{GL})$ et $a_i \in \mathbb{N}^*$
- **Caractère** : η attribue des signes ± 1 à chaque $\rho_i \boxtimes S_{a_i}$

(satisfaisant certaines conditions techniques)

Question

Étant donné $\pi = L(\mathfrak{m}, \phi, \eta) \in \text{Irr}(G)$, trouver $(\mathfrak{m}', \phi', \eta') \in \text{Data}(G)$ telles que

$$\hat{\pi} = L(\mathfrak{m}', \phi', \eta').$$

Symétrisation des données

- Pour $\Delta = [x, y]_\rho$, on définit $\Delta^\vee = [-y, -x]_{\rho^\vee}$
- Par linéarité, cela s'étend à Mult

Exemple : Pour $\mathbf{m} = [-1, 2] + [-1, 1]$ on a $\mathbf{m}^\vee = [-2, 1] + [-1, 1]$

Notons $\text{Symm} = \{\mathfrak{s} \in \text{Mult} \mid \mathfrak{s}^\vee = \mathfrak{s}\}$ (multisegments symétriques).

Définissons Symm^ε (multisegments symétriques avec des signes) comme les paires $(\mathfrak{s}, \varepsilon)$ où :

- $\mathfrak{s} \in \text{Symm}$
- ε attribue des signes ± 1 aux segments centrés (ceux avec $c(\Delta) = 0$)

Exemple : $\mathfrak{s} = [-1, 2] + [-2, 1] + [-1, 1]$ avec $\varepsilon([-1, 1]) = -1$

Des données de Langlands vers un multisegment symétrique :

$$\begin{aligned}\mathrm{Data}(G) &\longrightarrow \mathrm{Symm}^\varepsilon \\ (\mathfrak{m}, \phi, \eta) &\longmapsto (\mathfrak{s}, \varepsilon)\end{aligned}$$

où le multisegment symétrique est

$$\mathfrak{s} = \sum_{\Delta \in \mathfrak{m}} (\Delta + \Delta^\vee) + \sum_{\rho \boxtimes S_a \in \phi} \left[-\frac{a-1}{2}, \frac{a-1}{2} \right]_\rho$$

et les signes sont donnés par

$$\varepsilon \left(\left[-\frac{a-1}{2}, \frac{a-1}{2} \right]_\rho \right) = \eta(\rho \boxtimes S_a).$$

Notation : $\pi = L(\mathfrak{s}, \varepsilon)$

Donnée de Langlands :

$$(\mathfrak{m}, \phi, \eta) = ([-2, 1], S_1 + S_1 + S_3, (-1, -1, +1))$$

Exemple de l'application de transfert

Donnée de Langlands :

$$(\mathfrak{m}, \phi, \eta) = ([-2, 1], S_1 + S_1 + S_3, (-1, -1, +1))$$

Symétrisation :


$$\mathfrak{s} = [-2, 1] + [-1, 2] + [0, 0] + [0, 0] + [-1, 1]$$

avec $\varepsilon([0, 0]) = -1$ et $\varepsilon([-1, 1]) = +1$.

On définit une involution

$$\text{AD} : \text{Symm}^\varepsilon \longrightarrow \text{Symm}^\varepsilon$$

Théorème (L.-Mínguez)

Soit $\pi = L(\mathfrak{s}, \varepsilon) \in \text{Irr}(G)$. Alors

$$\widehat{\pi} = L(\text{AD}(\mathfrak{s}, \varepsilon)).$$

On définit une involution

$$\text{AD} : \text{Symm}^\varepsilon \longrightarrow \text{Symm}^\varepsilon$$

Théorème (L.-Mínguez)

Soit $\pi = L(\mathfrak{s}, \varepsilon) \in \text{Irr}(G)$. Alors

$$\widehat{\pi} = L(\text{AD}(\mathfrak{s}, \varepsilon)).$$

Remarque : Il n'est pas facile de trouver une involution sur Symm^ε !

Trois types de représentations cuspidales

- AD est défini sur chaque droite cuspidale $\mathbb{Z}_\rho$
- À la différence de GL_n , la cuspidale ρ importe !
- Trois cas différents selon ρ : Bon, Mauvais et Moche

AD est défini de façon **récursive** :

Pour $(\mathfrak{s}, \varepsilon) \in \text{Symm}^\varepsilon$, on construit :

- $(\mathfrak{s}_1, \varepsilon_1)$: premier multisegment du dual
- $(\mathfrak{s}^\#, \varepsilon^\#)$: nouveau multisegment symétrique

tel que

$$\text{AD}(\mathfrak{s}, \varepsilon) = (\mathfrak{s}_1, \varepsilon_1) + \text{AD}(\mathfrak{s}^\#, \varepsilon^\#)$$

Algorithme AD (cas « bon »)

Soit $(s, \varepsilon) \in \text{Symm}^\varepsilon$.

Étape 1 : Ordonner les segments selon un ordre spécifique $\preceq$.

Étape 2 : Construire une chaîne décroissante $\Delta_1 \succeq \dots \succeq \Delta_r$ où :

- Δ_1 est le segment maximal avec l'extrémité maximale
- Δ_{j+1} est le plus grand segment tel que :
 1. $\Delta_{j+1} \preceq \Delta_j$
 2. $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 3. **Condition de signe** : si $c(\Delta_{j+1}) = c(\Delta_j) = 0$ alors $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Étape 3 : Extraire s_1 à partir des $e(\Delta_j)$ et des $b(\Delta_j^\vee)$.

Étape 4 : Calculer $s^\#$ en retirant ces parties et ajuster les signes.

Exemple

Contexte : $G = \mathrm{Sp}_{2n}(F)$ et $\rho = 1$ (cas « bon »).

Donnée de Langlands :

$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Exemple

Contexte : $G = \mathrm{Sp}_{2n}(F)$ et $\rho = 1$ (cas « bon »).

Donnée de Langlands :

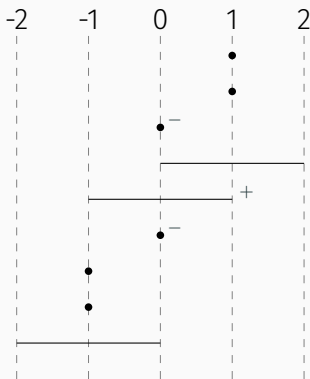
$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Symétrisation :

$$\mathfrak{s} = [-2, 0] + [0, 2] + [-1, -1] + [1, 1] + [-1, -1] + [1, 1] + [0, 0] + [0, 0] + [-1, 1]$$

avec $\varepsilon([0, 0]) = -1$ et $\varepsilon([-1, 1]) = 1$.

Exemple : étape par étape



Dual : $AD(\mathfrak{s}, \varepsilon) =$

...

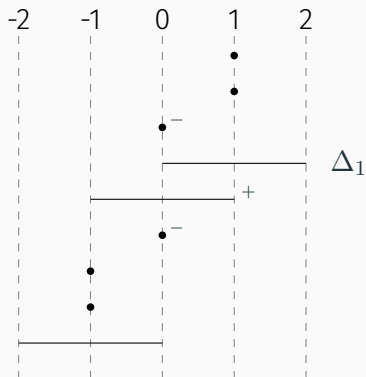
Rappel : algorithme AD

1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $AD(\mathfrak{s}, \varepsilon) =$

...

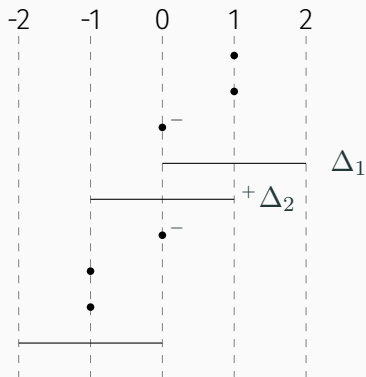
Rappel : algorithme AD

1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succcurlyeq \dots \succcurlyeq \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $AD(\mathfrak{s}, \varepsilon) =$

...

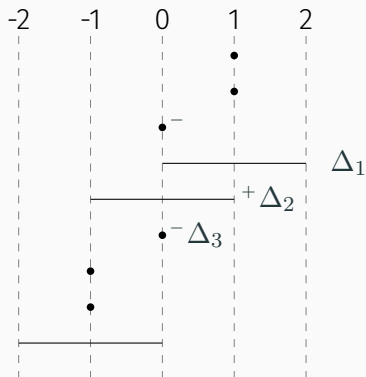
Rappel : algorithme AD

1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succcurlyeq \dots \succcurlyeq \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $AD(\mathfrak{s}, \varepsilon) =$

...

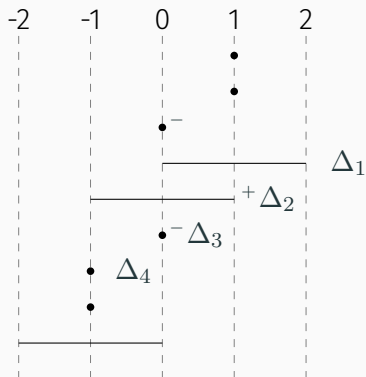
Rappel : algorithme AD

1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succcurlyeq \dots \succcurlyeq \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $AD(\mathfrak{s}, \varepsilon) =$

...

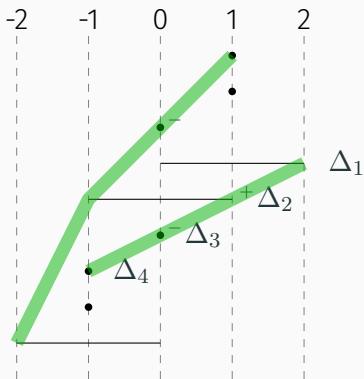
Rappel : algorithme AD

1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succ \dots \succ \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $AD(\mathfrak{s}, \varepsilon) =$
 $[-1, 2] + [-2, 1] + \dots$

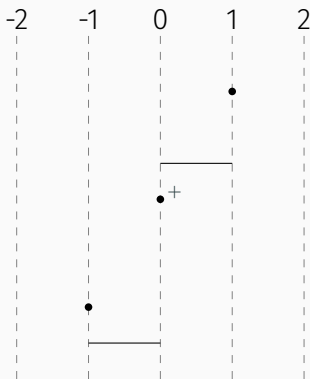
Rappel : algorithme AD

1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $AD(\mathfrak{s}, \varepsilon) =$
 $[-1, 2] + [-2, 1] + \dots$

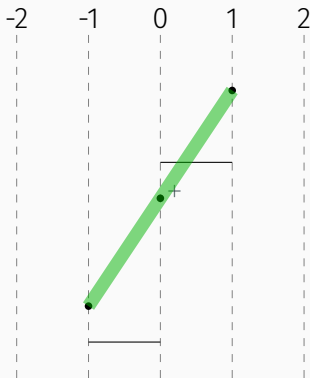
Rappel : algorithme AD

1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $\text{AD}(\mathfrak{s}, \varepsilon) =$

$$[-1, 2] + [-2, 1] + [-1, 1] + \dots$$

avec $\varepsilon([-1, 1]) = 1$

Rappel : algorithme AD

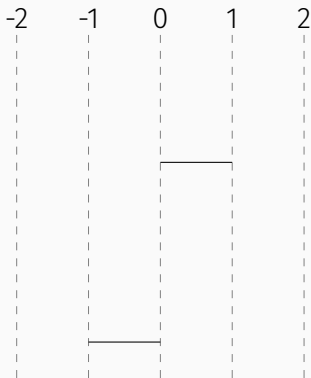
1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

$\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $AD(\mathfrak{s}, \varepsilon) =$
 $[-1, 2] + [-2, 1] + [-1, 1] + \dots$
avec $\varepsilon([-1, 1]) = 1$

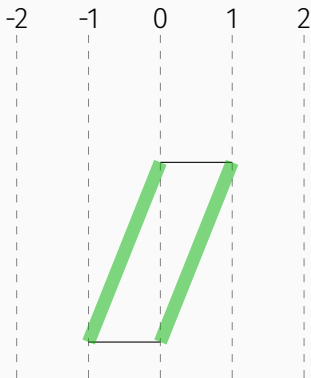
Rappel : algorithme AD

1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $\text{AD}(\mathfrak{s}, \varepsilon) =$

$$[-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0]$$

avec $\varepsilon([-1, 1]) = 1$

Rappel : algorithme AD

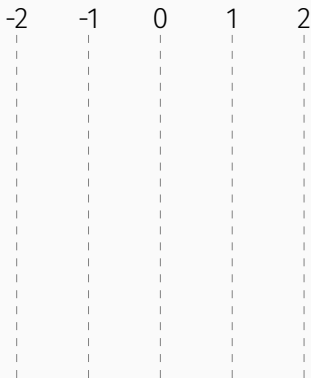
1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succ \dots \succ \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

$\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Exemple : étape par étape



Dual : $\text{AD}(\mathfrak{s}, \varepsilon) =$

$$[-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0]$$

avec $\varepsilon([-1, 1]) = 1$

Rappel : algorithme AD

1. Ordonner selon $\preceq$
2. Δ_1 = plus grand segment avec $e(\Delta_1)$ maximal
3. $\Delta_1 \succ \dots \succ \Delta_l$ inductivement :
 Δ_{j+1} plus grand avec
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Résultat :

$\mathfrak{s}_1$ = segment formé des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

$\mathfrak{m}^\#$ = suppression des $e(\Delta_i)$ et $b(\Delta_j^\vee)$

Entrée :

$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Exemple : Résultat

Entrée :

$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Symétriser + appliquer AD :

$$\text{AD}(\mathfrak{s}, \varepsilon) = ([-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0], \varepsilon')$$

avec $\varepsilon'([-1, 1]) = 1$.

Exemple : Résultat

Entrée :

$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Symétriser + appliquer AD :

$$\text{AD}(\mathfrak{s}, \varepsilon) = ([-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0], \varepsilon')$$

avec $\varepsilon'([-1, 1]) = 1$.

Données de Langlands du dual d'Aubert :

$$\hat{\pi} = L([-2, 1] + [-1, 0], S_3, (1))$$

Atobe et Mínguez (2023) ont développé un algorithme pour calculer la dualité d'Aubert–Zelevinsky en utilisant les ρ -dérivées.

Atobe et Mínguez (2023) ont développé un algorithme pour calculer la dualité d'Aubert-Zelevinsky en utilisant les ρ -dérivées.

$$\mathbf{m} = [-2, 0] + 2[-1, -1]$$

$$\phi = S_1 + S_1 + S_3$$

$$\eta = (-1, -1, 1)$$

Atobe et Mínguez (2023) ont développé un algorithme pour calculer la dualité d'Aubert-Zelevinsky en utilisant les ρ -dérivées.

$$\begin{array}{ll} \mathbf{m} = [-2, 0] + 2[-1, -1] & D_{|\cdot|^{-1}} \mathbf{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 & \longrightarrow \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) & \eta = (-1, -1, 1) \end{array}$$

Atobe et Mínguez (2023) ont développé un algorithme pour calculer la dualité d'Aubert-Zelevinsky en utilisant les ρ -dérivées.

$$\begin{array}{ccccc} \mathbf{m} = [-2, 0] + 2[-1, -1] & D_{|\cdot|-1} & \mathbf{m} = [0, -2] & D_{L([-1, 0])} & \mathbf{m} = [-2, -2] \\ \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) \end{array}$$

Atobe et Mínguez (2023) ont développé un algorithme pour calculer la dualité d'Aubert-Zelevinsky en utilisant les ρ -dérivées.

$$\begin{array}{ccccccc} \mathfrak{m} = [-2, 0] + 2[-1, -1] & D_{|\cdot|-1} & \mathfrak{m} = [0, -2] & D_{L([-1, 0])} & \mathfrak{m} = [-2, -2] & D_{|\cdot|-2} & \mathfrak{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) \end{array}$$

Atobe et Mínguez (2023) ont développé un algorithme pour calculer la dualité d'Aubert-Zelevinsky en utilisant les ρ -dérivées.

$$\begin{array}{ccccccc}
 \mathfrak{m} = [-2, 0] + 2[-1, -1] & D_{|\cdot|-1} & \mathfrak{m} = [0, -2] & D_{L([-1, 0])} & \mathfrak{m} = [-2, -2] & D_{|\cdot|-2} & \mathfrak{m} = \emptyset \\
 \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 \\
 \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) \\
 & & & & & & \downarrow \pi \mapsto \hat{\pi} \\
 & & & & & & \mathfrak{m} = [0, -1] \\
 & & & & & & \phi = S_1 \\
 & & & & & & \eta = (1)
 \end{array}$$

Atobe et Mínguez (2023) ont développé un algorithme pour calculer la dualité d'Aubert-Zelevinsky en utilisant les ρ -dérivées.

$$\begin{array}{ccccccc}
 \begin{array}{l} \mathfrak{m} = [-2, 0] + 2[-1, -1] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-1}} & \begin{array}{l} \mathfrak{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{L([-1, 0])}} & \begin{array}{l} \mathfrak{m} = [-2, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-2}} & \begin{array}{l} \mathfrak{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} \\
 & & & & & & \downarrow \pi \mapsto \hat{\pi} \\
 & & & & \begin{array}{l} \mathfrak{m} = [0, -2] \\ \phi = S_1 \\ \eta = (1) \end{array} & \xleftarrow{S_{|\cdot|^2}} & \begin{array}{l} \mathfrak{m} = [0, -1] \\ \phi = S_1 \\ \eta = (1) \end{array}
 \end{array}$$

Atobe et Mínguez (2023) ont développé un algorithme pour calculer la dualité d'Aubert-Zelevinsky en utilisant les ρ -dérivées.

$$\begin{array}{ccccccc}
 \begin{array}{l} \mathfrak{m} = [-2, 0] + 2[-1, -1] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-1}} & \begin{array}{l} \mathfrak{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{L([-1, 0])}} & \begin{array}{l} \mathfrak{m} = [-2, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-2}} & \begin{array}{l} \mathfrak{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} \\
 & & & & & & \downarrow \pi \mapsto \hat{\pi} \\
 & & \begin{array}{l} \mathfrak{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xleftarrow{S_{Z([0, 1])}} & \begin{array}{l} \mathfrak{m} = [0, -2] \\ \phi = S_1 \\ \eta = (1) \end{array} & \xleftarrow{S_{|\cdot|^2}} & \begin{array}{l} \mathfrak{m} = [0, -1] \\ \phi = S_1 \\ \eta = (1) \end{array}
 \end{array}$$

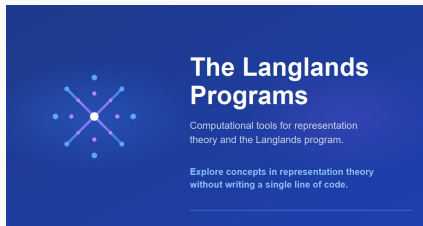
Atobe et Mínguez (2023) ont développé un algorithme pour calculer la dualité d'Aubert-Zelevinsky en utilisant les ρ -dérivées.

$$\begin{array}{ccccccc}
 \begin{array}{l} \mathbf{m} = [-2, 0] + 2[-1, -1] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-1}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{L([-1, 0])}} & \begin{array}{l} \mathbf{m} = [-2, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-2}} & \begin{array}{l} \mathbf{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} \\
 & & & & & & \downarrow \pi \mapsto \hat{\pi} \\
 \begin{array}{l} \mathbf{m} = [0, -1] + [1, -2] \\ \phi = S_3 \\ \eta = (1) \end{array} & \xleftarrow{S_{|\cdot|}^{(2)}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xleftarrow{S_{Z([0, 1])}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 \\ \eta = (1) \end{array} & \xleftarrow{S_{|\cdot|^2}} & \begin{array}{l} \mathbf{m} = [0, -1] \\ \phi = S_1 \\ \eta = (1) \end{array}
 \end{array}$$

Implémentation

Implémentations :

- Paquet Python/Sage :
`github.com/ThomasLanard/aubert-zelevinsky-duality`
- Application web (avec Petar Bakić et Elad Zelingher) :
`langlandsprograms.com`



Machine Learning

Motivation :

- L'IA excelle à trouver des motifs dans de grands ensembles de données.
- En mathématiques, générer des données est facile.
- L'IA peut être utilisée pour découvrir de nouvelles relations entre des objets mathématiques.

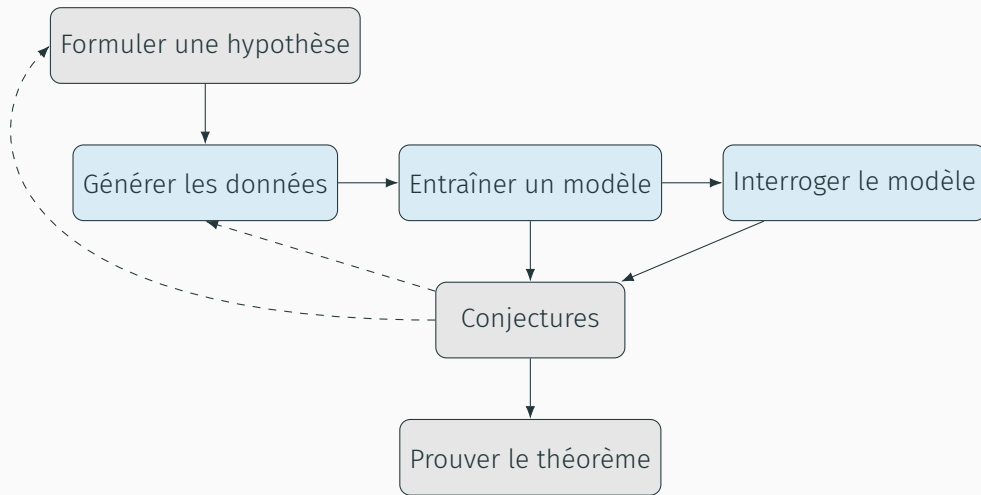
Motivation :

- L'IA excelle à trouver des motifs dans de grands ensembles de données.
- En mathématiques, générer des données est facile.
- L'IA peut être utilisée pour découvrir de nouvelles relations entre des objets mathématiques.

Dans notre travail :

- Nous utilisons l'apprentissage supervisé, en nous appuyant sur l'algorithme d'Atobe-Mínguez, pour développer une intuition sur la dualité d'Aubert-Zelevinsky.
- Nous avons commencé ce processus sans faire aucune hypothèse ou conjecture mathématique, afin d'explorer quelles structures le modèle d'IA pourrait révéler de lui-même.

Flux de travail machine learning



Un **réseau de neurones** (neural network) est une fonction $\varphi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ donnée par composition :

$$\varphi = \varphi_r \circ \cdots \circ \varphi_1$$

où chaque **couche** (layer) est :

$$\varphi_i : \mathbb{R}^{n_i} \xrightarrow{\text{linéaire}} \mathbb{R}^{n_{i+1}} \xrightarrow{\text{ReLU}} \mathbb{R}^{n_{i+1}}$$

ReLU (Rectified Linear Unit) : $x \mapsto \max(0, x)$ appliquée composante par composante

(pas de ReLU sur la dernière couche φ_r)

Objectif : Approcher une fonction inconnue $\hat{\varphi} : \mathbb{R}^n \rightarrow \mathbb{R}^m$

Données d'entraînement : échantillons $(x_i, \hat{\varphi}(x_i))_{i=1,\dots,N}$

Fonction de perte : mesure l'erreur de prédiction

$$l(\varphi) = \frac{1}{N} \sum_{i=1}^N \|\varphi(x_i) - \hat{\varphi}(x_i)\|^2$$

Entraînement : utiliser la **descente de gradient** pour trouver les paramètres qui minimisent $l(\varphi)$

Question

Peut-on prédire directement le dual complet $AD(\mathfrak{m}, \phi, \eta)$?

Question

Peut-on prédire directement le dual complet $AD(\mathbf{m}, \phi, \eta)$?

Résultat

Le modèle montre une **précision faible**.

Première tentative : prédire le dual complet

Question

Peut-on prédire directement le dual complet $AD(\mathbf{m}, \phi, \eta)$?

Résultat

Le modèle montre une **précision faible**.

Nouvelle stratégie : Plutôt que de prédire tout le dual en une fois, essayer de prédire **un segment**.

Question

Quel segment devons-nous prédire en premier ?

Merci pour votre attention !